

APIs with Node.js and Express.js for Shoppyglobe E-commerce

POST /login

Purpose: Retrieve users with a token.

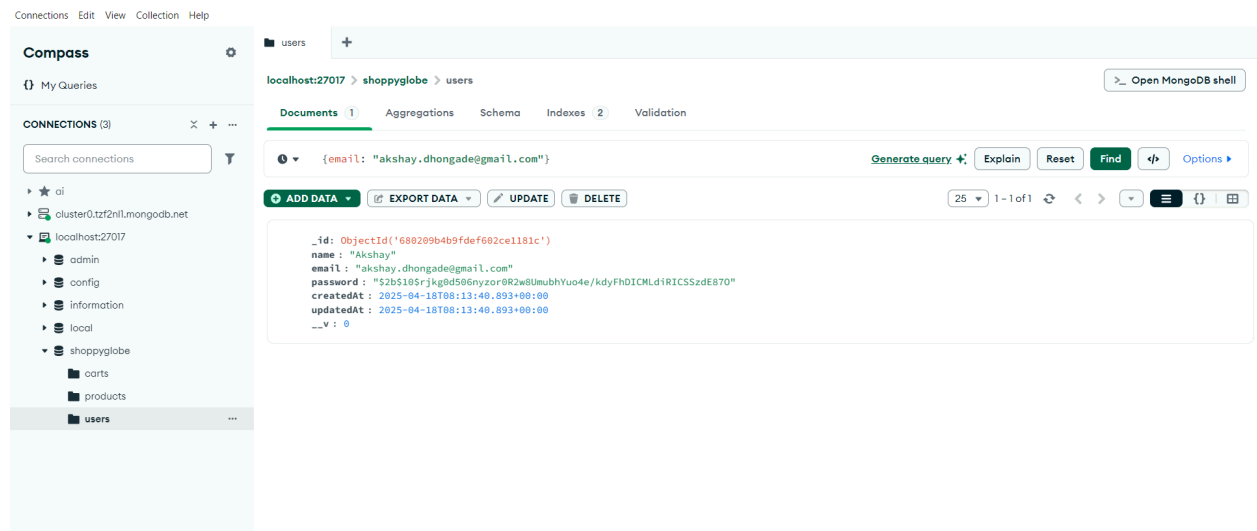
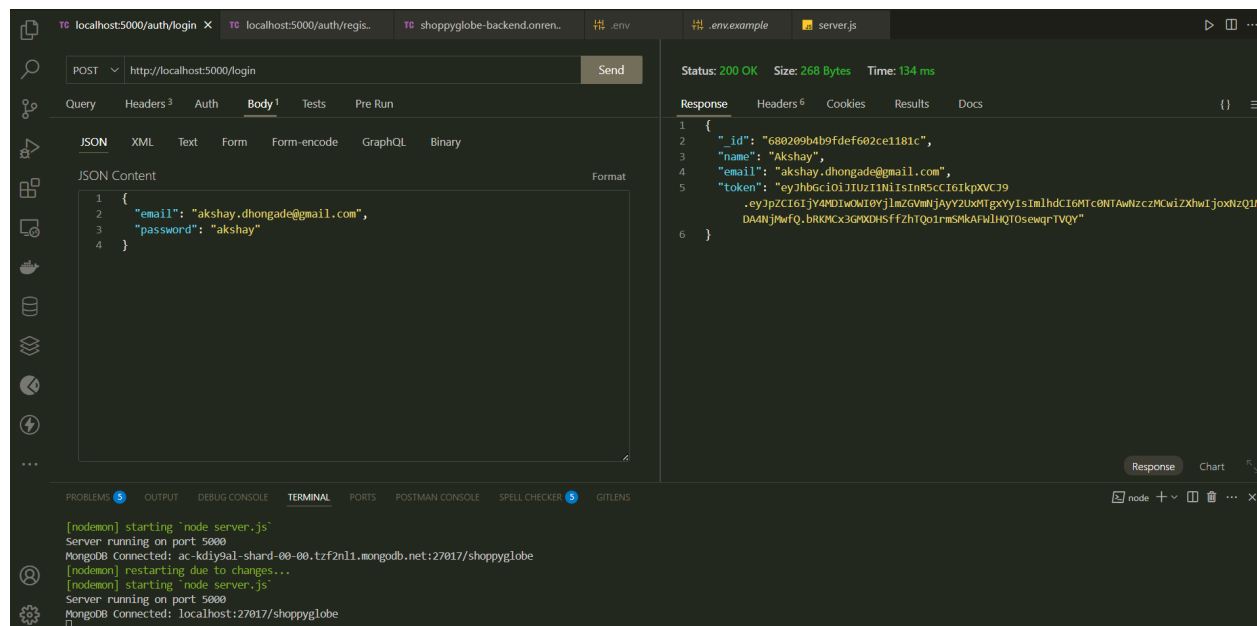
Request Type: POST

Endpoint: /login

Expected Response: 200 OK

Response Body: Object with user properties

Screenshot:



POST /register

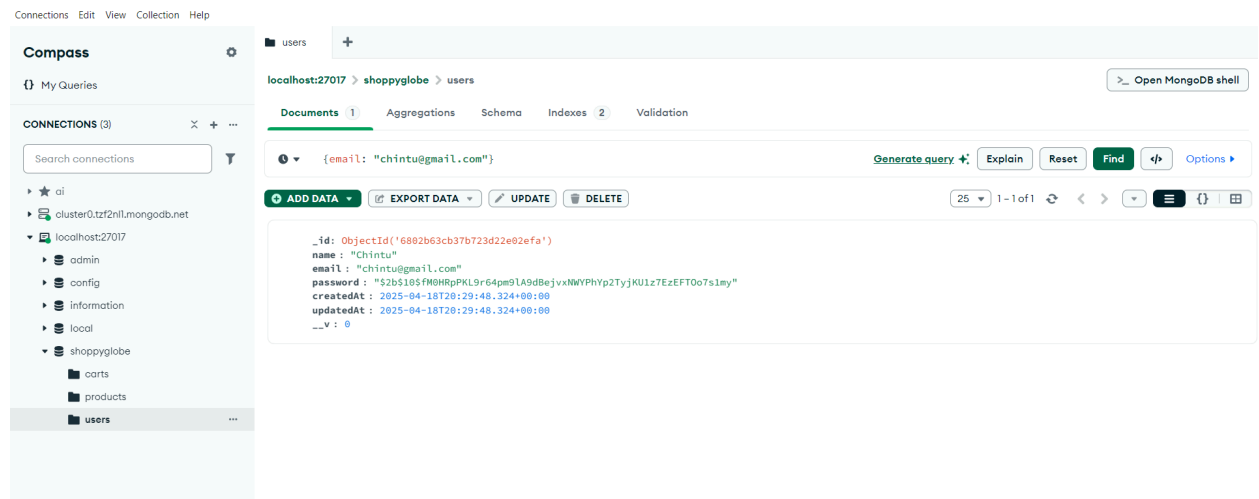
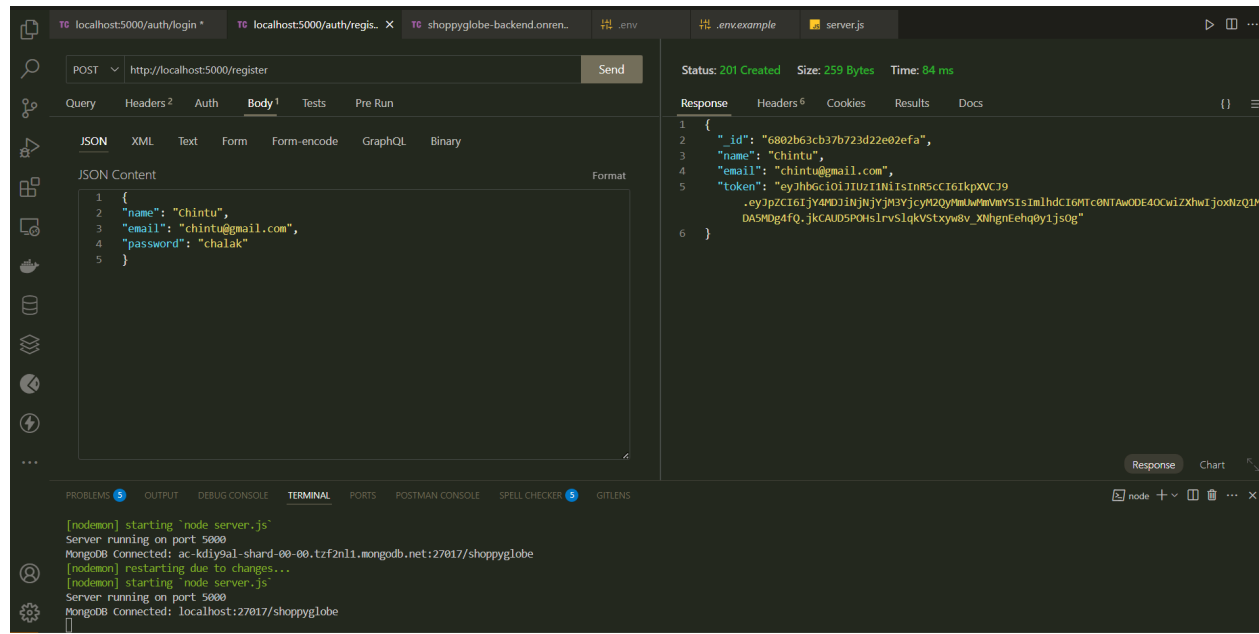
Purpose: Register user.

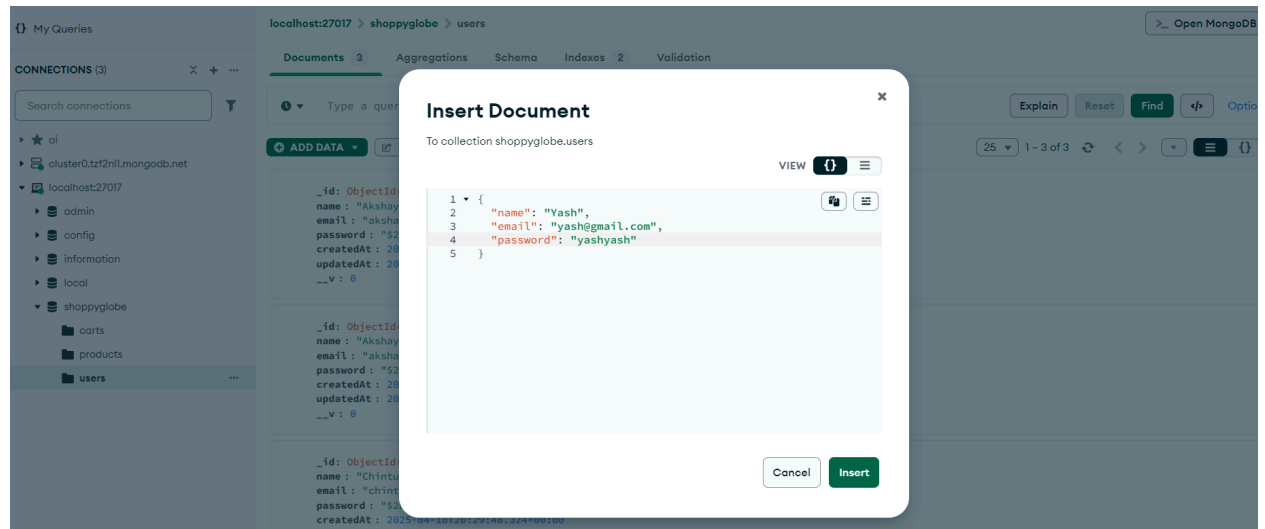
Request Type: POST

Endpoint: /register

Expected Response: 201 Created

Screenshot:





```
_id: ObjectId('6802b8850f01b7323cfb263d')
name: "Yash"
email: "yash@gmail.com"
password: "yashyash"
```

GET /products

Purpose: Get all products list.

Request Type: GET

Endpoint: /products

Expected Response: 200 Ok

Screenshot:

The screenshot shows the Postman interface with a GET request to `http://localhost:5000/products/` sent successfully. The status is 200 OK, size is 4.62 KB, and time is 50 ms. The response body is a JSON array of two product objects.

```
1 [
2   {
3     "_id": "6801148f035abaebfac035ec",
4     "name": "Essence Mascara Lash Princess",
5     "price": 9.99,
6     "description": "The Eyeshadow Palette with Mirror offers a versatile range of
7       eyeshadow shades for creating stunning eye looks. With a built-in mirror,
8       it's convenient for on-the-go makeup application.",
9     "stock": 15,
10    "images": [
11      "https://cdn.dummyjson.com/products/images/beauty/
12        Essence%20Mascara%20Lash%20Princess/1.png"
13    ],
14    "_v": 0,
15    "createdAt": "2025-04-17T14:47:43.237Z",
16    "updatedAt": "2025-04-17T14:47:43.237Z"
17  },
18  {
19    "_id": "6801148f035abaebfac035ed",
20    "name": "Eyeshadow Palette with Mirror",
21    "price": 19.99,
22    "description": "The Eyeshadow Palette with Mirror offers a versatile range of
23      eyeshadow shades for creating stunning eye looks. With a built-in mirror,
24      it's convenient for on-the-go makeup application.",
25    "stock": 15,
26    "images": [
27      "https://cdn.dummyjson.com/products/images/beauty
```

The screenshot shows the MongoDB Compass interface. The 'products' collection is selected, and three documents are displayed in the list view.

```
{ "_id": ObjectId("6801148f035abaebfac035ec"),
  "name": "Essence Mascara Lash Princess",
  "price": 9.99,
  "description": "The Eyeshadow Palette with Mirror offers a versatile range of eyeshado...",
  "stock": 15,
  "images": Array (1)
  ...v: 0
  createdAt: 2025-04-17T14:47:43.237+00:00
  updatedAt: 2025-04-17T14:47:43.237+00:00
}
```

```
{ "_id": ObjectId("6801148f035abaebfac035ed"),
  "name": "Eyeshadow Palette with Mirror",
  "price": 19.99,
  "description": "The Eyeshadow Palette with Mirror offers a versatile range of eyeshado...",
  "stock": 15,
  "images": Array (1)
  ...v: 0
  createdAt: 2025-04-17T14:47:43.239+00:00
  updatedAt: 2025-04-17T14:47:43.239+00:00
}
```

```
{ "_id": ObjectId("6801148f035abaebfac035ee"),
  "name": "Powder Canister",
  "price": 14.99,
  "description": "The Powder Canister is a finely milled setting powder designed to set _",
  "stock": 15,
  "images": Array (1)
  ...v: 0
  createdAt: 2025-04-17T14:47:43.239+00:00
  updatedAt: 2025-04-17T14:47:43.239+00:00
}
```

GET /products/:id

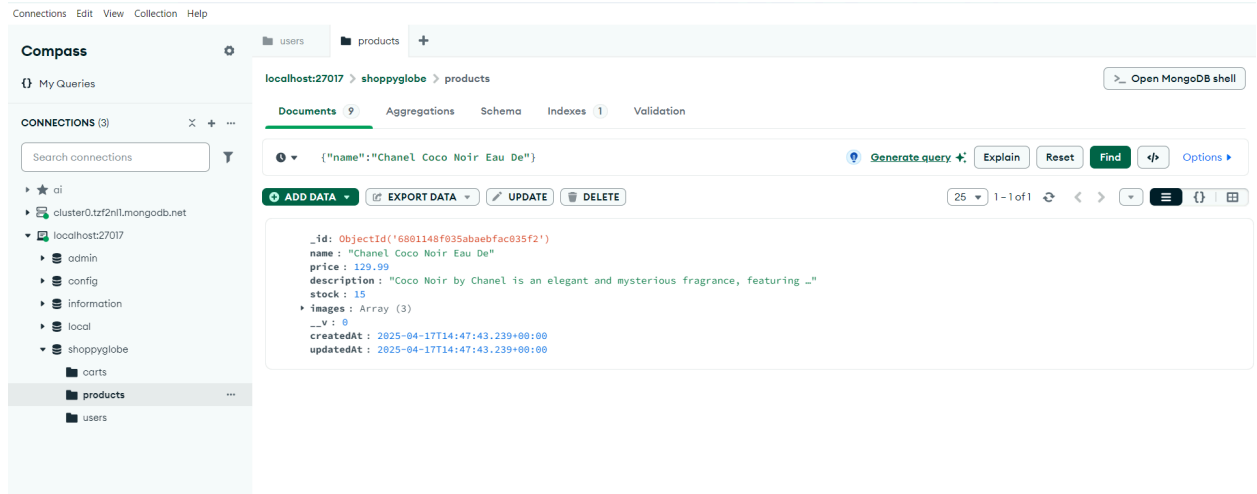
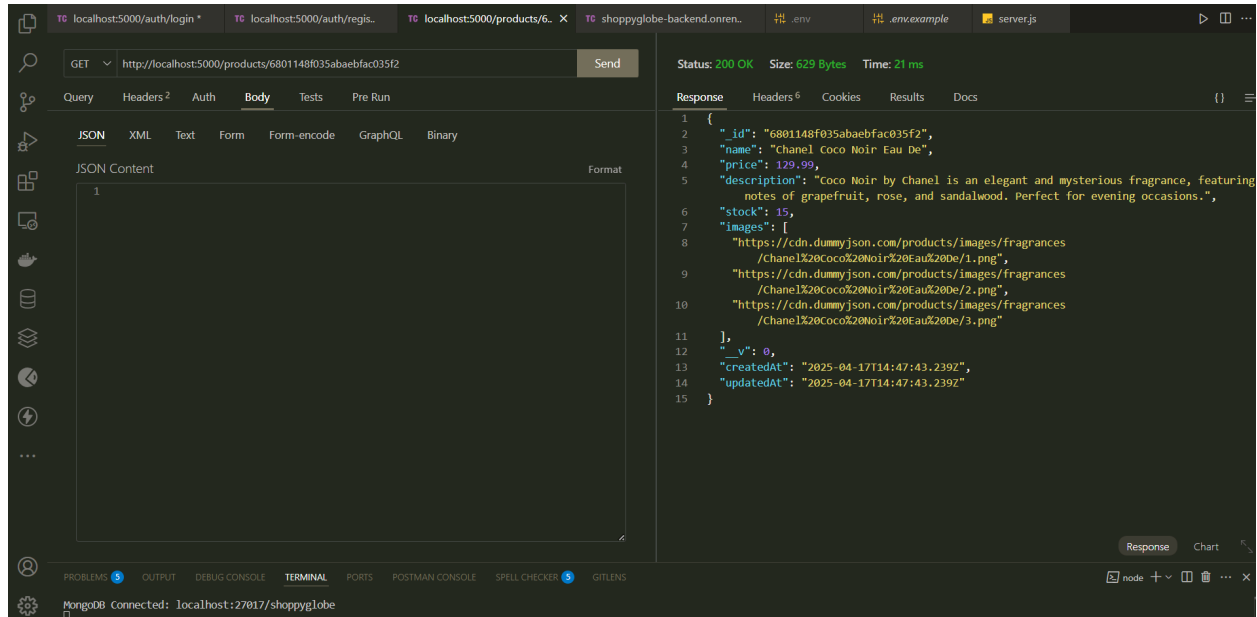
Purpose: Get a product details

Request Type: GET

Endpoint: /products/:id

Expected Response: 200 OK

Screenshot:



GET /cart

Purpose: Get a cart list

Request Type: GET

Endpoint: /cart

Expected Response: 200 OK

Screenshot:

Postman interface showing a GET request to `http://localhost:5000/cart`. The response is a JSON object with status 200 OK, size 819 Bytes, and time 30 ms. The JSON contains user information and a list of items in the cart.

```
{
  "_id": "6802c6a2b37b723d22e02f0e",
  "user": "680209b4b9fdef602ce1181c",
  "items": [
    {
      "product": {
        "_id": "6801148f035abaebfac035f2",
        "name": "Chanel Coco Noir Eau De",
        "price": 129.99,
        "description": "Coco Noir by Chanel is an elegant and mysterious fragrance, featuring notes of grapefruit, rose, and sandalwood. Perfect for evening occasions.",
        "stock": 15,
        "images": [
          "https://cdn.dummyjson.com/products/images/fragrances/Chanel%20Coco%20Noir%20Eau%20De/1.png",
          "https://cdn.dummyjson.com/products/images/fragrances/Chanel%20Coco%20Noir%20Eau%20De/2.png",
          "https://cdn.dummyjson.com/products/images/fragrances/Chanel%20Coco%20Noir%20Eau%20De/3.png"
        ],
        "_v": 0,
        "createdAt": "2025-04-17T14:47:43.239Z",
        "updatedAt": "2025-04-17T14:47:43.239Z"
      },
      "quantity": 1
    }
  ]
}
```

MongoDB Compass interface showing the `carts` collection. The collection contains two documents. The first document has an empty `items` array, and the second document has one item in the cart.

```
{
  "_id": ObjectId("68011ce04f2a218e6efe277e"),
  "user": ObjectId("680118d2228e4f3e011bdalc"),
  "items": Array (empty),
  "createdAt": 2025-04-17T15:23:12.514+00:00,
  "updatedAt": 2025-04-17T19:51:45.111+00:00,
  "_v": 7
}
```

```
{
  "_id": ObjectId("6802c6a2b37b723d22e02f0e"),
  "user": ObjectId("680209b4b9fdef602ce1181c"),
  "items": Array (1),
  "createdAt": 2025-04-18T21:39:46.217+00:00,
  "updatedAt": 2025-04-18T21:39:46.217+00:00,
  "_v": 0
}
```

POST /cart

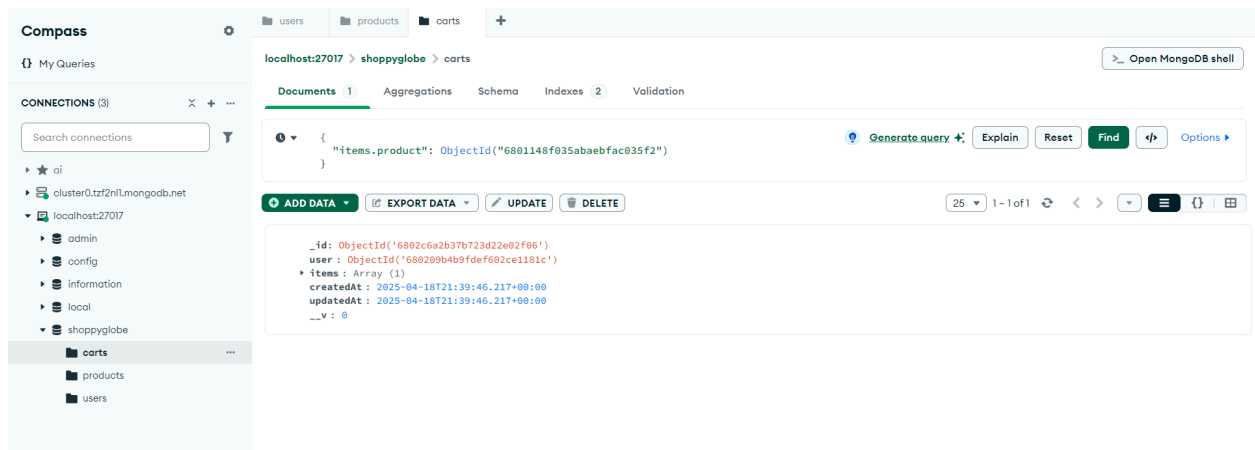
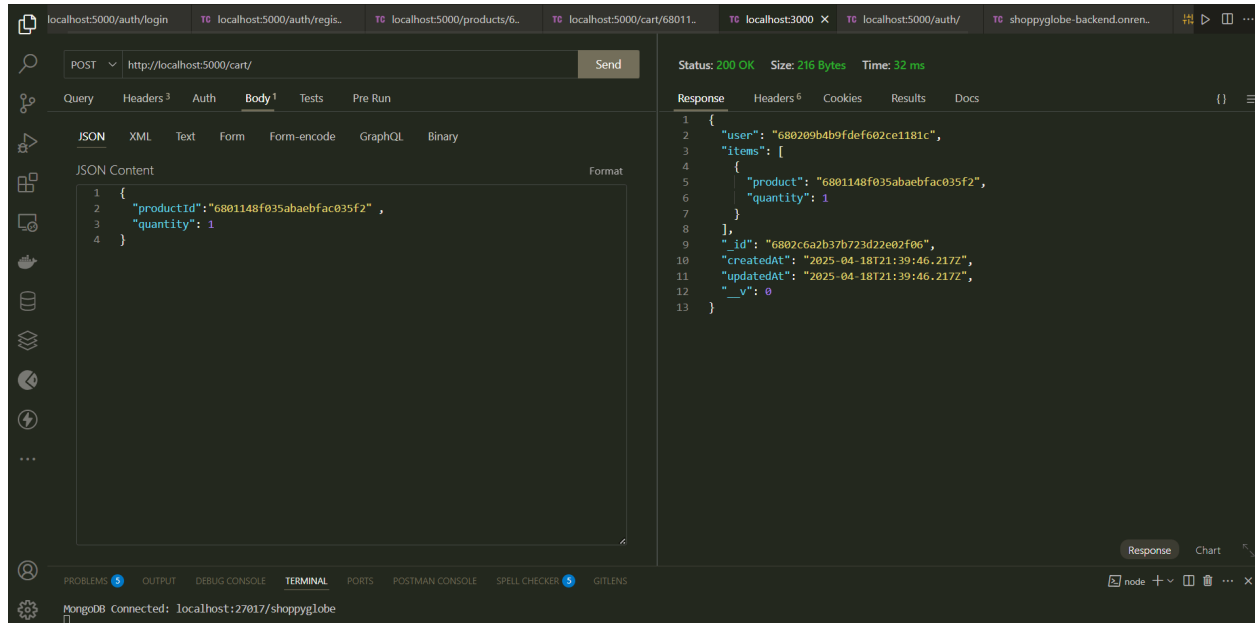
Purpose: Add to cart

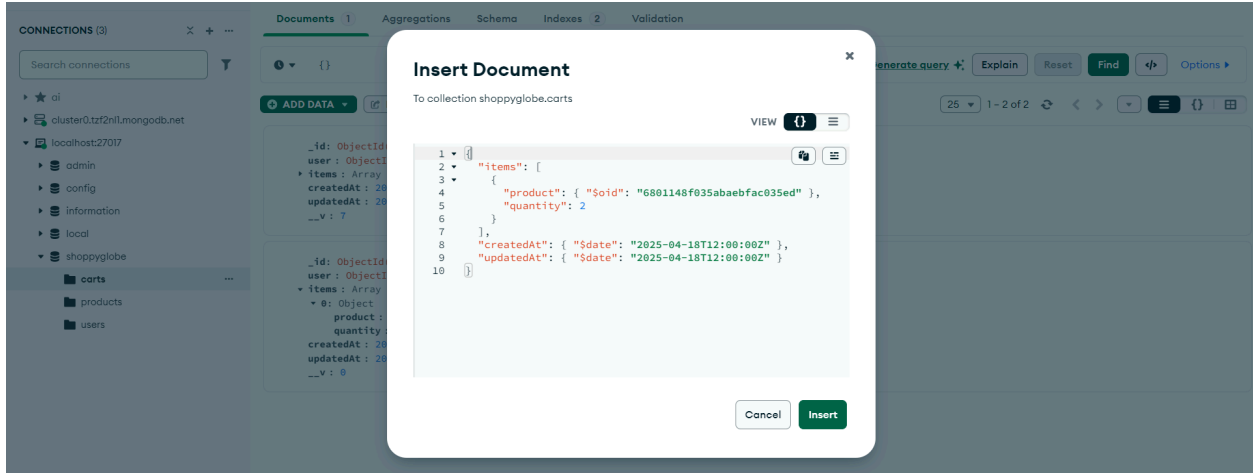
Request Type: POST

Endpoint: /cart

Expected Response: 200 OK

Screenshot:





PUT /cart/:id

Purpose: Add to cart

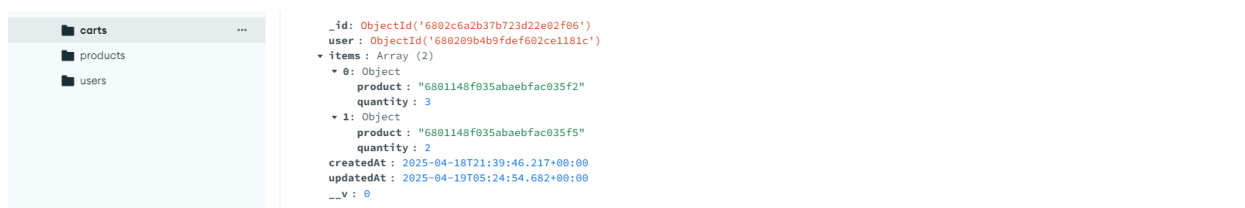
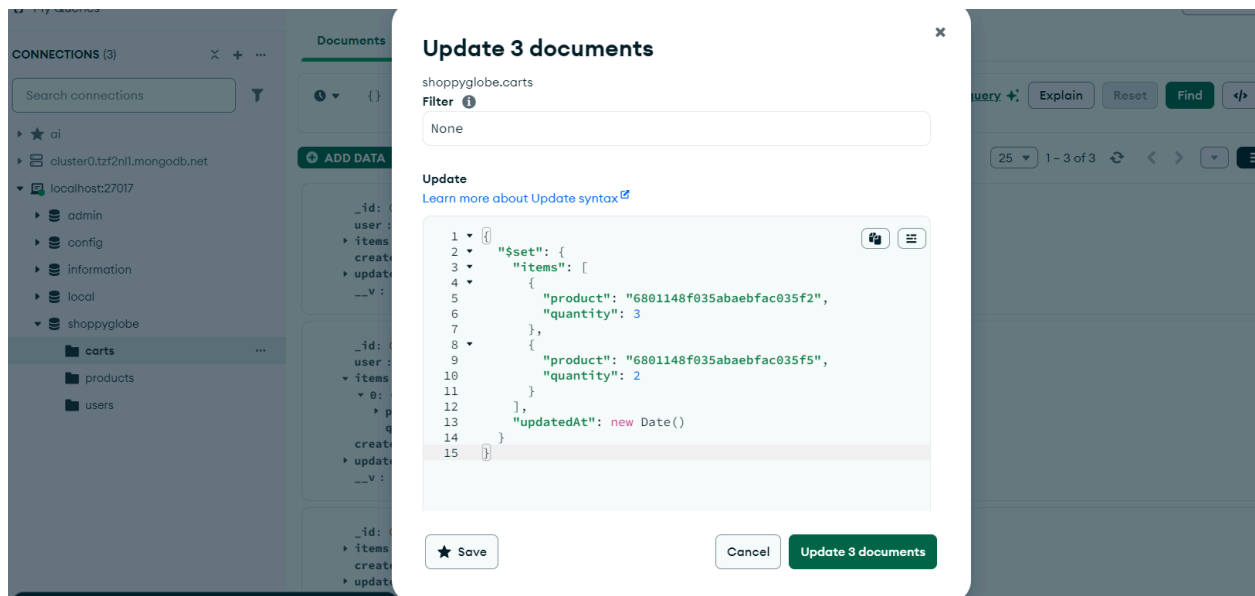
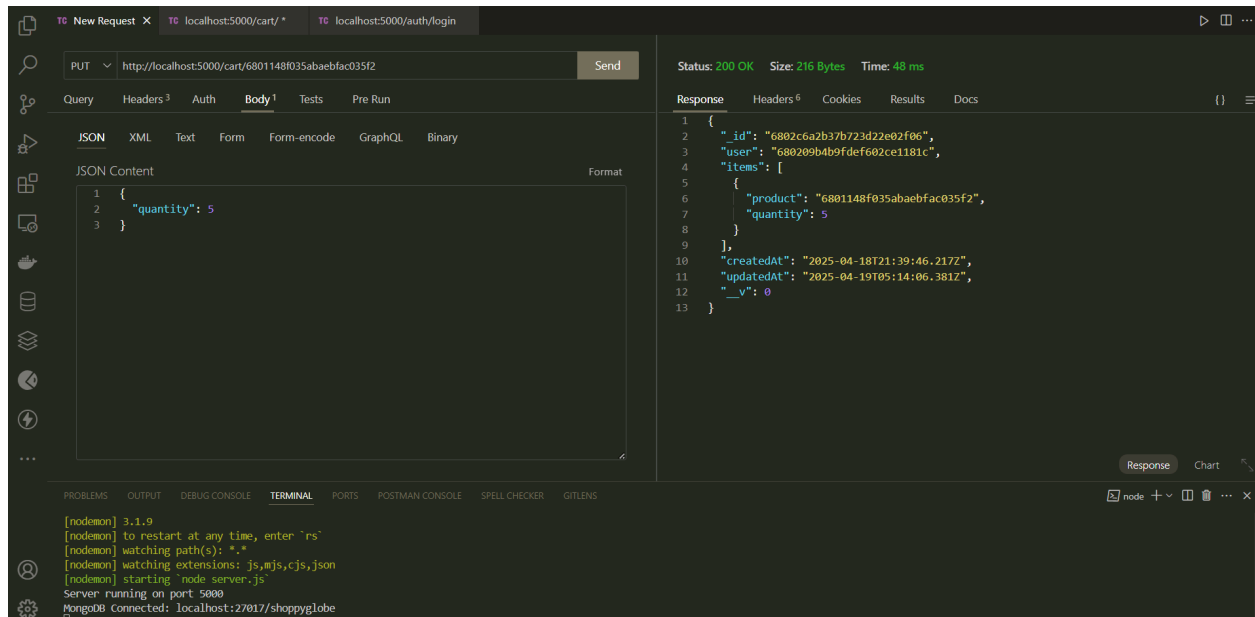
Request Type: PUT

Endpoint: /cart/:id

Expected Response: 200 OK

Response Body: object with quantity property

Screenshot:



DELETE /cart/:id

Purpose: Remove a cart item by their ID.

Request Type: DELETE

Endpoint: /cart/:id

Expected Response: 200 OK

Screenshot:

The screenshot displays a Postman interface for a DELETE request and its response, along with a MongoDB database view.

Postman Request:

- Method: DELETE
- URL: `http://localhost:5000/cart/6801148f035abaebfac035f2`
- Headers:
 - Accept: */*
 - User-Agent: Thunder Client (https://www.thunderclient.com)
 - Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZiZC

Postman Response:

```
1 {
2   "message": "Item removed from cart successfully",
3   "cart": {
4     "_id": "6802c6a2b37b723d22e02f06",
5     "user": "680209b4b9fdef602ce1181c",
6     "items": [
7       {
8         "product": "6801148f035abaebfac035f5",
9         "quantity": 2
10      }
11    ],
12    "createdAt": "2025-04-18T21:39:46.217Z",
13    "updatedAt": "2025-04-19T05:30:07.997Z",
14    "__v": 1
15  }
16 }
```

Terminal Output:

```
[nodemon] 3.1.9
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running on port 5000
MongoDB Connected: localhost:27017/shoppyglobe
```

MongoDB Database View:

- Database: shoppyglobe
- Collection: carts
- Document:

```
{
  "_id": ObjectId('6802c6a2b37b723d22e02f06'),
  "user": ObjectId('680209b4b9fdef602ce1181c'),
  "items": [
    {
      "product": ObjectId('6801148f035abaebfac035f5'),
      "quantity": 2
    }
  ],
  "createdAt": 2025-04-18T21:39:46.217+00:00,
  "updatedAt": 2025-04-19T05:30:07.997+00:00,
  "__v": 1
}
```

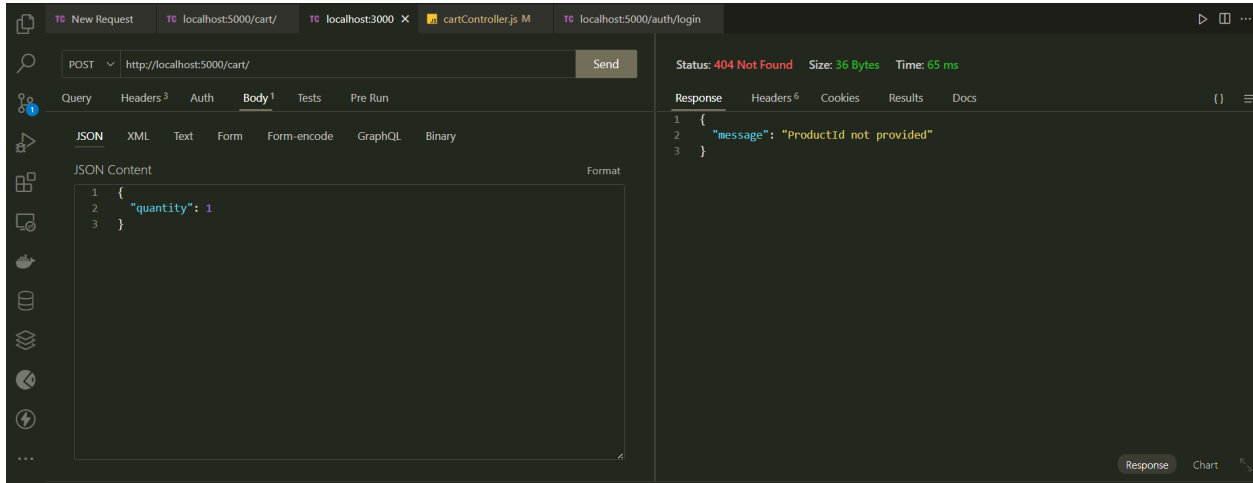
Error Handling Screenshots

1. VALIDATE INPUT (POST) /cart without required fields

Expected: 400 Bad Request

Message: "ProductId not provided"

Screenshot:

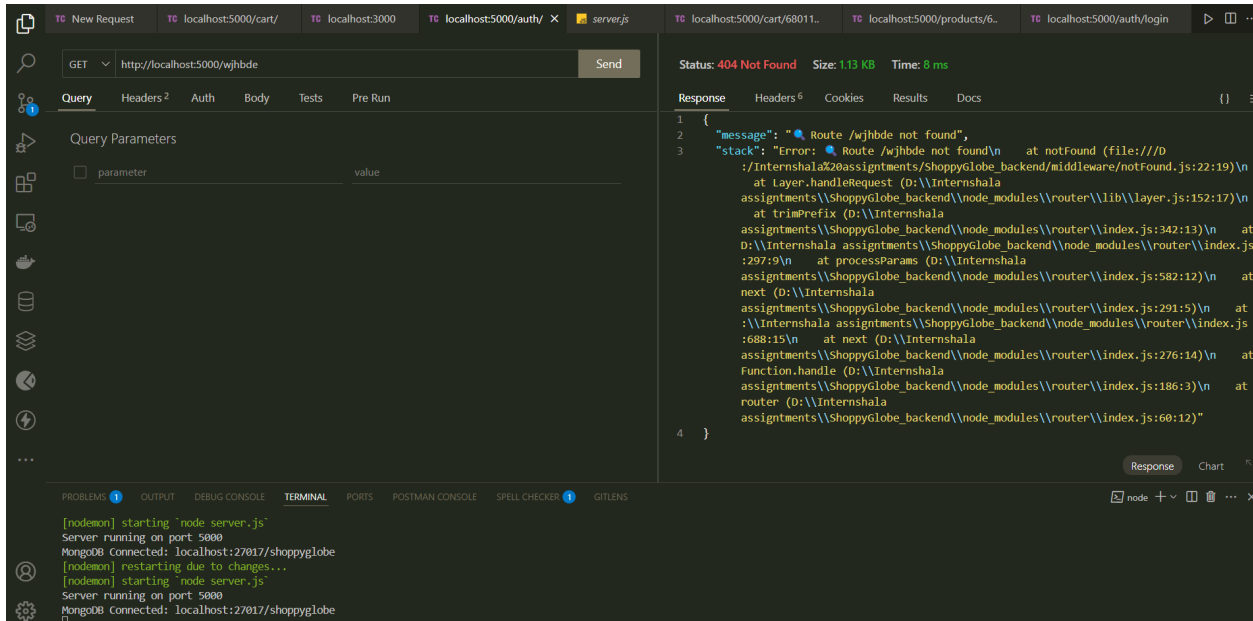


2. INVALID ROUTE (GET/PUT/DELETE) with invalid route

Expected: 404 Not Found

Message: "Route /jend not found"

Screenshot:

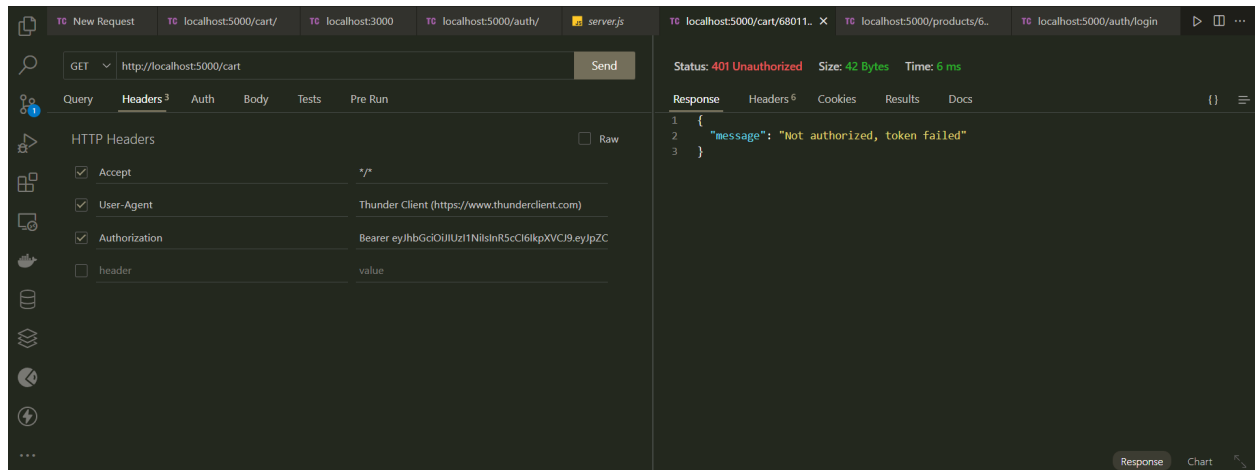


3. INVALID TOKEN (GET,PUT,POST,DELETE) /cart with invalid ID

Expected: 401 Unauthorized

Message: "Not authorized, token failed"

Screenshot:



4. INVALID CREDENTIALS /login with invalid credentials

Expected: 400 Bad Request

Message: "Invalid credentials"

Screenshot:

